

HOMEWORK 1

>>NAME HERE<<

>>ID HERE<<

Instructions:

Use this LaTeX file as a template to develop your homework and submit it as a single PDF file on Gradescope. For the programming component, you can use any programming language, but all learning algorithms need to be implemented from scratch (i.e. no `scikit-learn`). Put all code used for the assignment at the end of the document in whatever way is easiest but still legible.

1 A Simple Decision Tree

Implement a simple decision-tree learner for classification that works for data of the following type:

- each item has two continuous features $\mathbf{x} \in \mathbb{R}^2$
- the class label is binary and encoded as $y \in \{0, 1\}$
- data files are in plaintext with one labeled item per line, separated by whitespace:

$$\begin{array}{ccc} x_{11} & x_{12} & y_1 \\ & \dots & \\ x_{n1} & x_{n2} & y_n \end{array}$$

Your program should implement a decision tree learner according to the following guidelines:

- Candidate splits (j, c) for numeric features should use a threshold c in feature dimension j in the form of $x_{.j} \geq c$.
- c should be on values of that dimension present in the training data; i.e. the threshold is on training points, not in between training points. You may enumerate all features, and for each feature, use all possible values for that dimension.
- You may skip those candidate splits with zero split information (i.e. the entropy of the split), and continue the enumeration.
- The left branch of such a split is the “then” branch, and the right branch is “else”.
- Splits should be chosen using information gain ratio. If there is a tie you may break it arbitrarily.
- The stopping criteria (for making a node into a leaf) are that
 - the node is empty, or
 - all splits have zero gain ratio (if the entropy of the split is non-zero), or
 - the entropy of any candidates split is zero
- To simplify, whenever there is no majority class in a leaf, let it predict $y = 1$.

1.1 Stopping at pure labels [10 points]

If a node is not empty but contains training items with the same label, why is it guaranteed to become a leaf? Explain. You may assume that the feature values of these items are not all the same.

The decision tree works by trying to maximize the information gain or minimizing the entropy. If a node is not empty but has training items has the same labels, the entropy is zero. Also, since the feature values of these items are not all the same, the entropy of the split is non-zero. Trying to create a split will give an information gain ratio of zero. Since information gain ratio is zero, the algorithm stops and all items are classified under a single label in that node. (Points are given for mentioning in any form that information gain is 0.)

1.2 Greediness [5 points]

Handcraft a small training set where both classes are present but the algorithm refuses to split; instead it makes the root a leaf and stop; Importantly, if we were to manually force a split, the algorithm will happily continue splitting the data set further and produce a deeper tree with zero training error. You should (1) plot your training set, (2) explain why. Hint: you don't need more than a handful of items.

Half points for plotting the data and half for the explanation. Any dataset is accepted as long as the explanation is correct and the dataset is legible. The data points should be such that the information gain should be zero irrespective of the split. This suggests that the data should be such that for any root split, the both nodes should have equal number of items of both class. This would make the information gain ratio zero, and the algorithm will stop at the root. An example of this data could simply be 4 points (0,0,0),(1,0,1),(0,1,1),(1,1,0). (Any other data that gives an information gain ratio of zero is okay.) If the splits are chosen as $x_1 \geq 0$ or $x_2 \geq 0$, the entropy would still be one. It remains 1 even if the splits chosen $x_1 \geq 1$ or $x_2 \geq 1$. Hence, the root becomes a leaf node. Now, if we force a split at say $x_1 \geq 1$, then the algorithm proceeds further with an information gain of 1 and ends with zero training error.

1.3 Information gain [5 points]

Use the training set `Drums.txt`. For the root node, list all candidate cuts and their information gain ratio. If the entropy of the candidate split is zero, please list its mutual information (i.e. information gain). Hint: to get $\log_2(x)$ when your programming language may be using a different base, use $\log(x) / \log(2)$. Also, please follow the split rule in the first section.

Being a very small straightforward dataset, the candidate cuts and gain ratio will be similar.

Dimension	Candidate Cut	Gain Ratio / Mutual Information
1	0.1	0.100518
1	0	0
2	-2	0
2	-1	0.100518
2	0	0.055954
2	1	0.005780
2	2	0.001144
2	3	0.016411
2	4	0.049749
2	5	0.111240
2	6	0.236100
2	7	0.055954
2	8	0.430157

1.4 The king of interpretability [5 points]

Decision tree is not the most accurate classifier in general. However, it persists. This is largely due to its rumored interpretability: a data scientist can easily explain a tree to a non-data scientist. Build a tree from `D3leaves.txt`. Then manually convert your tree to a set of logic rules. Show the tree¹ and the rules.

Half the marks are for the tree and other half for explaining the tree. Different logic rules are allowed. If some kind of explanation is given and is correct but the logic rules are not given, 2 points will be deducted. It's ok for students to just give either the answer in Figure 1 or the one in Figure 2.

1.5 Or is it? [10 points]

For this question only, make sure you DO NOT VISUALIZE the data sets or plot your tree's decision boundary in the 2D x space. If your code does that, turn it off before proceeding. This is because you want to see your own reaction when trying to interpret a tree. You will get points no matter what your interpretation is. And we will ask you to visualize them in the next question anyway.

¹When we say "show the tree" you can either use the standard CS tree view or some crude plaintext representation of the tree – as long as you explain the format. When we say "visualize the tree" we mean a plot in the 2D x space that shows how the tree will classify any points.

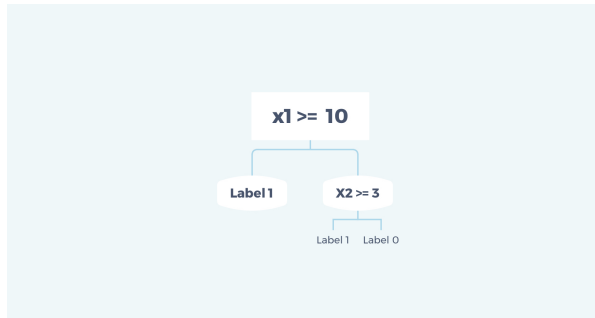
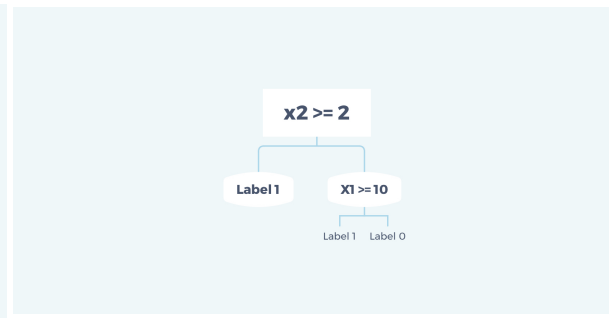
Figure 1: $Y \iff (x_1 \geq 10) \vee (x_2 \geq 3)$ Figure 2: $Y \iff (x_2 \geq 2) \vee (x_1 \geq 10)$ 

Figure 3: Tree for Question 1.5.1

1. Build and show a decision tree on `D1.txt`. Again, do not visualize the data set or the tree in the x input space. In real tasks you will not be able to visualize the whole high dimensional input space anyway, so we don't want you to "cheat" here.
2. Look at your tree in the above format (remember, you should not visualize the 2D dataset or your tree's decision boundary) and try to interpret the decision boundary in human understandable English.
The tree can be interpreted to mean that all items labelled 1 are above a line $x_2 = 0.20183$ and all items labeled 0 are below this line. A straight line parallel to an axis separates the data.
3. Build a decision tree on `D2.txt`. Show it to us.
The tree is very complicated with around 61 nodes. Points are given if the tree is similar enough to the expected one. If the number of nodes is the same order as ours, full points are given. Otherwise if the tree is given and the depth is off, some points will be taken off.
4. Try to interpret the decision tree trained on `D2.txt`. Is it easy or possible to do so without visualization?
This tree is very difficult to interpret without visualizing it in space.

1.6 Hypothesis space visualization [5 points]

For `D1.txt` and `D2.txt`, do the following separately:

1. Produce a scatter plot of the data set.
2. Visualize your decision tree's decision boundary (or decision region, or some other ways to clearly visualize how your decision tree will make decisions in the feature space).

Then discuss why the size of your decision trees on `D1.txt` and `D2.txt` differ. Relate this to the hypothesis space of our decision tree algorithm.

The data in `D1.txt` is clearly separated by a line parallel to one of the axis. On the other hand, the data in `D2.txt` is separated by a slanting line which is not parallel to any axis. In the case of `D2.txt`, one split along the line parallel to the axis is enough to split the data. To simulate the line parallel to the axis, the algorithm makes a lot of small splits along the axis. Since there are a lot of cuts, the tree is pretty deep.

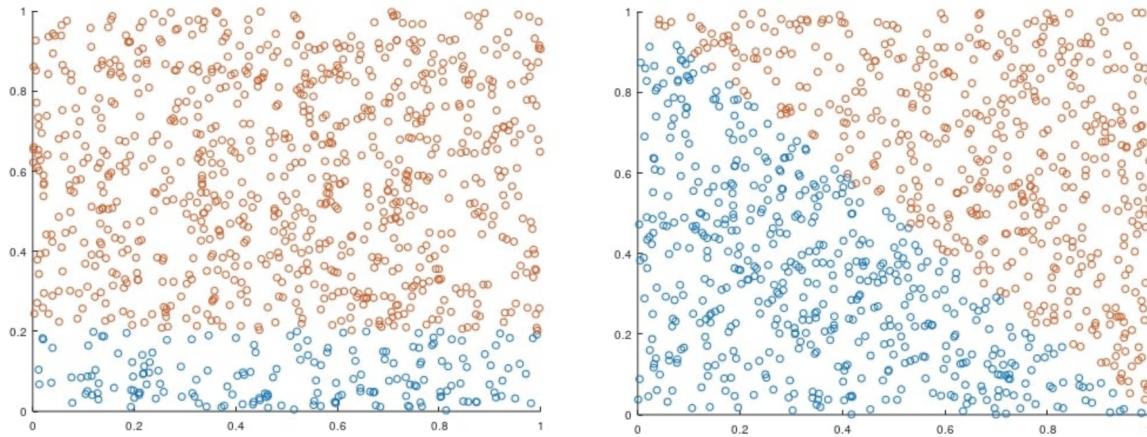


Figure 4: Question 1.6.1

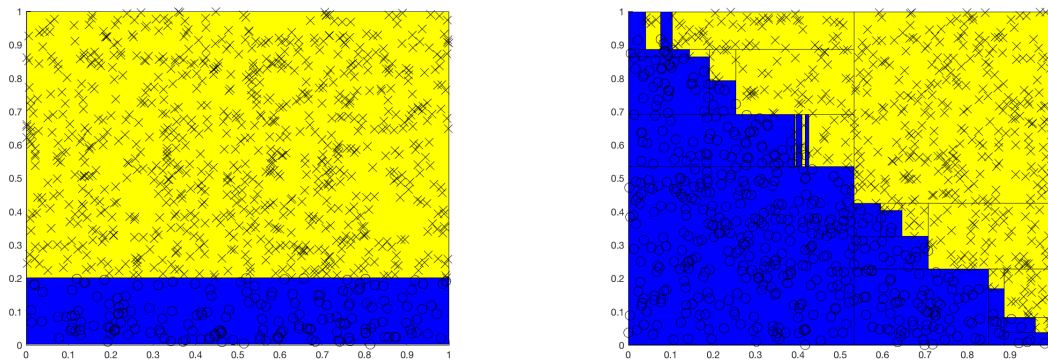


Figure 5: Question 1.6.2

1.7 Learning curve [10 points]

We provide a data set `Dbig.txt` with 10000 labeled items. Caution: `Dbig.txt` is sorted. Randomly split `Dbig.txt` into a candidate training set of 8192 items and a test set (the rest). Do this by generating a random permutation and splitting at 8192. Then, generate a sequence of five nested training sets $D_{32} \subset D_{128} \subset D_{512} \subset D_{2048} \subset D_{8192}$ from the candidate training set. Here the subscript n in D_n denotes training set size. The easiest way is to take the first n items from the (same) permutation from before. This sequence simulates the real world situation where you obtain more and more training data. Finally, for each D_n , train a decision tree, measure its test set error err_n , and show three things in your answer:

1. A three-column table with a column showing n , a column showing the number of nodes in the n th tree, and a column showing err_n
2. Plot n vs. err_n . This is known as a learning curve (a single plot).
3. Visualize your decision trees' decision boundary (five plots).

3 points for the (1), 2 points for (2) and 1 point each for each graph in (3). The number of nodes should be in the same order as the solution. A point will be taken off for unreasonable number of nodes. The error should be within 5% of numbers given in Figure 6. Points will be deducted for large errors. The graph should resemble the one given in Figure 7. The decision boundaries should be approximately similar to the ones shown in Figure 8. Points will be deducted for a boundary which looks very different.

	1	2	3
1	32	9	0.1350
2	128	19	0.1090
3	512	49	0.0697
4	2048	125	0.0326
5	8192	257	0.0100

Figure 6: Question 1.7.1

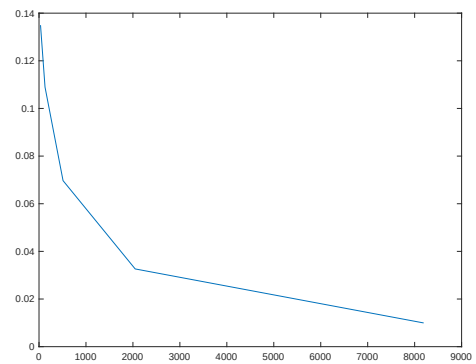


Figure 7: Question 1.7.2

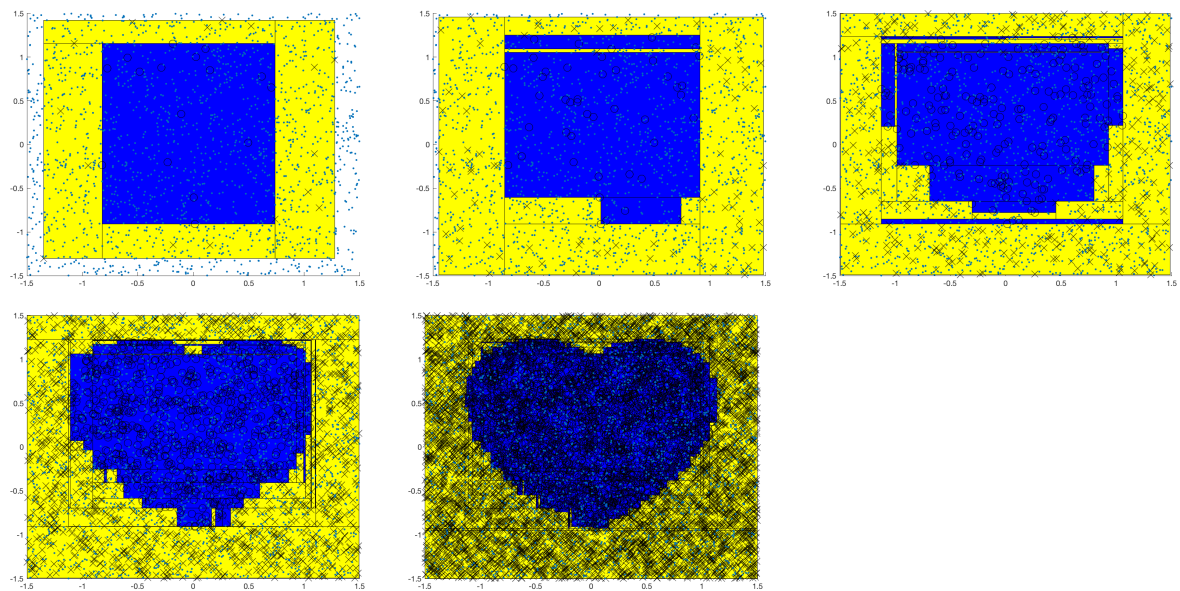


Figure 8: Question 1.7.3

2 Maximum Likelihood Estimation

This problem explores maximum likelihood estimation (MLE), which is a technique for estimating an unknown parameter of a probability distribution based on observed samples. Suppose we observe the values of n iid² random variables $X_1, \dots, X_n$ drawn from a single Bernoulli distribution with parameter θ . In other words, for each X_i , we know that

$$P(X_i = 1) = \theta \quad \text{and} \quad P(X_i = 0) = 1 - \theta.$$

Our goal is to estimate the value of θ from these observed values of X_1 through X_n .

For any hypothetical value $\hat{\theta}$, we can compute the probability of observing the outcome $X_1, \dots, X_n$ if the true parameter value θ were equal to $\hat{\theta}$. This probability of the observed data is often called the *data likelihood*, and the function $L(\hat{\theta}) = P(X_1, \dots, X_n | \hat{\theta})$ that maps each $\hat{\theta}$ to the corresponding likelihood is called the *likelihood function*. A natural way to estimate the unknown parameter θ is to choose the $\hat{\theta}$ that maximizes the likelihood function. Formally,

$$\hat{\theta}^{\text{MLE}} = \arg \max_{\hat{\theta}} L(\hat{\theta}).$$

Often it is more convenient to work with the log likelihood function $\ell(\hat{\theta}) = \log L(\hat{\theta})$. Since the log function is increasing, we also have

$$\hat{\theta}^{\text{MLE}} = \arg \max_{\hat{\theta}} \ell(\hat{\theta}).$$

2.1 Bernoulli likelihood [5 points]

Write a formula for the log likelihood function, $\ell(\hat{\theta})$. Your function should depend on $N_1 = \#\{X_i = 1\}$ and $N_0 = \#\{X_i = 0\}$ and the hypothetical parameter $\hat{\theta}$, and should be simplified as far as possible (i.e. don't just write the definition of the log likelihood function).

$$\begin{aligned} \ell(\hat{\theta}) &= \log \left(P(X_1, \dots, X_n | \hat{\theta}) \right) \\ &= \log \left(\prod_{i=1}^n P(X_i | \hat{\theta}) \right) \\ &= \sum_{i=1}^n \log P(X_i | \hat{\theta}) \\ &= \sum_{i=1}^n \log \hat{\theta}^{X_i} (1 - \hat{\theta})^{1-X_i} \\ &= N_1 \log \hat{\theta} + N_0 \log(1 - \hat{\theta}) \end{aligned}$$

2.2 Bernoulli example [5 points]

Consider the following sequence of 10 samples:

$$X = (0, 1, 0, 1, 1, 0, 0, 1, 1, 1).$$

Compute the maximum likelihood estimate for the 10 samples. Show all of your work (hint: recall that if x^* maximizes $f(x)$, then $f'(x^*) = 0$).

Since the log function is always increasing, the $\hat{\theta}$ that maximizes the log likelihood is the same as the $\hat{\theta}$ that maximizes the likelihood. The first and second derivatives of ℓ are given by: $\ell'(\hat{\theta}) = \frac{N_1}{\hat{\theta}} - \frac{N_0}{1-\hat{\theta}}$ and $\ell''(\hat{\theta}) = -\left(\frac{N_1}{\hat{\theta}^2} + \frac{N_0}{(1-\hat{\theta})^2}\right)$. Since the second derivative is always negative, the function ℓ is concave, and we can find its maximizer by solving $\ell'(\hat{\theta}) = 0$. The solution to this equation is obtained by straightforward algebra and is given by $\theta^{\text{MLE}} = \frac{N_1}{N_1 + N_0}$. For the given dataset, we compute $\theta^{\text{MLE}} = 0.6$.

²independent & identically distributed.

2.3 Binomial likelihood [5 points]

Now we will consider a related distribution. Suppose we observe the values of m iid random variables $Y_1, \dots, Y_m$ drawn from a single Binomial distribution $B(n, \theta)$. A Binomial distribution models the number of 1's from a sequence of n independent Bernoulli variables with parameter θ . In other words,

$$P(Y_i = k) = \binom{n}{k} \theta^k (1 - \theta)^{n-k} = \frac{n!}{k!(n-k)!} \cdot \theta^k (1 - \theta)^{n-k}.$$

Write a formula for the log likelihood function, $\ell(\hat{\theta})$. Your function should depend on $S = \sum_{i=1}^m Y_i$ and $T = \sum_{i=1}^m \log \binom{n}{Y_i}$ and the hypothetical parameter $\hat{\theta}$.

$$\begin{aligned} \ell(\hat{\theta}) &= \log P(Y_1, \dots, Y_m | \hat{\theta}) \\ &= \log \prod_{i=1}^m P(Y_i | \hat{\theta}) \\ &= \sum_{i=1}^m \log P(Y_i | \hat{\theta}) \\ &= \sum_{i=1}^m \log \binom{n}{Y_i} \hat{\theta}^{Y_i} (1 - \hat{\theta})^{n-Y_i} \\ &= \sum_{i=1}^m \log \binom{n}{Y_i} + Y_i \log \hat{\theta} + (n - Y_i) \log(1 - \hat{\theta}) \\ &= S \log \hat{\theta} + (mn - S) \log(1 - \hat{\theta}) + T \end{aligned}$$

2.4 Binomial example [5 points]

Consider two Binomial random variables Y_1 and Y_2 with the same parameters, $n = 5$ and θ . The Bernoulli variables for Y_1 and Y_2 resulted in $(0, 1, 0, 1, 1)$ and $(0, 0, 1, 1, 1)$, respectively. Therefore, $Y_1 = 3$ and $Y_2 = 3$. Compute the maximum likelihood estimate for the 2 samples. Show your work.

Since the log function is always increasing, the $\hat{\theta}$ that maximizes the log likelihood is the same as the $\hat{\theta}$ that maximizes the likelihood. The first and second derivatives of l are given by $\ell'(\hat{\theta}) = \frac{S}{\hat{\theta}} - \frac{mn-S}{1-\hat{\theta}}$ and $\ell''(\hat{\theta}) = -\left(\frac{S}{\hat{\theta}^2} - \frac{mn-S}{(1-\hat{\theta})^2}\right)$

Since the second derivative is always negative, the function l is concave, and we can find its maximizer by solving $\ell'(\hat{\theta}) = 0$. The solution to this equation is obtained by straightforward algebra and is given by $\hat{\theta}^{MLE} = \frac{\sum_{i=1}^m k_i}{mn}$

For the given dataset, we compute $\hat{\theta}^{MLE} = 0.6$

2.5 Comparison [5 points]

How do your answers for parts 1 and 3 compare? What about parts 2 and 4? If you got the same or different answers, why was that the case?

The answers to Part 2 and 4 are the same because a binomial random variable is the sum of bernoulli random variables and the data provided in those parts had the same total number of successes. The answers to Part 1 and 3 are almost the same whenever $\sum_{i=1}^m Y_i = N_1$ and $N_1 + N_0 = nm$ but the order of the individual Bernoulli trials can matter in Part 3 due to the ' n choose Y_i ' term.

3 Logistic regression

In this question, we will study the logistic regression algorithm.

3.1 Conditional likelihood [10 points]

For simplicity, we assume the dataset is two-dimensional. Given a training set $\{(x^i, y^i), i = 1, \dots, n\}$ where $x^i \in \mathbb{R}^2$ is a feature vector and $y^i \in \{0, 1\}$ is a binary label, we want to find the parameters $\hat{w}$ that maximize the likelihood for the training set, assuming a parametric model of the form

$$p(y = 1 | x; w) = \frac{1}{1 + \exp(-w_0 - w_1 x_1 - w_2 x_2)} = \frac{\exp(w_0 + w_1 x_1 + w_2 x_2)}{1 + \exp(w_0 + w_1 x_1 + w_2 x_2)}. \quad (1)$$

Below, we give a derivation of the conditional log likelihood. In this derivation, **provide a short justification for why each line follows from the previous one.**

$$\ell(w) \equiv \ln \prod_{j=1}^n p(y^j | x^j, w) \quad (2)$$

$$= \sum_{j=1}^n \ln p(y^j | x^j, w) \text{ follows from the fact, } \log(X \cdot Y) = \log X + \log Y \quad (3)$$

$$= \sum_{j=1}^n \ln \left(p(y^j = 1 | x^j, w)^{y^j} p(y^j = 0 | x^j, w)^{1-y^j} \right) \quad (4)$$

we show $p(y | x) = p(y = 1 | x)^y p(y = 0 | x)^{1-y}$ by considering two cases.

If $y = 1$, $p(y | x) = p(y = 1 | x) = p(y = 1 | x)^1 p(y = 0 | x)^0$.

If $y = 0$, $p(y | x) = p(y = 0 | x) = p(y = 1 | x)^0 p(y = 0 | x)^1$.

$$= \sum_{j=1}^n [y^j \ln p(y^j = 1 | x^j, w) + (1 - y^j) \ln p(y^j = 0 | x^j, w)] \quad (5)$$

using the logarithm of product and exponent rules.

$$= \sum_{j=1}^n \left[y^j \ln \frac{\exp(w_0 + w_1 x_1^j + w_2 x_2^j)}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} + (1 - y^j) \ln \frac{1}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} \right] \quad (6)$$

Plugging in Equation 1 and using $p(y = 0 | x, w) = 1 - p(y = 1 | x, w)$

$$= \sum_{j=1}^n \left[y^j \ln \left(\exp(w_0 + w_1 x_1^j + w_2 x_2^j) \right) + \ln \left(\frac{1}{1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j)} \right) \right] \quad (7)$$

using algebra and the logarithm of a product rule.

$$= \sum_{j=1}^n \left[y^j \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) - \ln \left(1 + \exp(w_0 + w_1 x_1^j + w_2 x_2^j) \right) \right]. \quad (8)$$

using the definition of logarithm, and the logarithm of a product rule.

3.2 Deriving the gradient [10 points]

Next, we will derive the gradient of the previous expression with respect to w_0, w_1, w_2 , i.e., $\frac{\partial \ell(w)}{\partial w_i}$, where $\ell(w)$ denotes the log likelihood from part 1. We will perform a few steps of the derivation, and then ask you to do one step at the end. If we take the derivative of Expression 8 with respect to w_i for $i \in \{1, 2\}$, we get the following expression:

$$\frac{\partial \ell(w)}{\partial w_i} = \frac{\partial}{\partial w_i} \sum_{j=1}^n \left[y^j \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right] - \frac{\partial}{\partial w_i} \sum_{j=1}^n \ln \left[1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right]. \quad (9)$$

The blue expression is linear in w_i , so it can be simplified to $\sum_{j=1}^n y^j x_i^j$. For the red expression, we use the chain rule as follows (first we consider a single $j \in [1, n]$)

$$\frac{\partial}{\partial w_i} \ln \left[1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right] \quad (10)$$

$$= \frac{1}{1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right)} \cdot \frac{\partial}{\partial w_i} \left(1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right) \quad (11)$$

$$= \frac{1}{1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right)} \cdot \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \frac{\partial}{\partial w_i} \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \quad (12)$$

$$= x_i^j \cdot \frac{\exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right)}{1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right)} \quad (13)$$

Now use Equation 13 (and the previous discussion) to show that overall, Expression 9, i.e., $\frac{\partial \ell(w)}{\partial w_i}$, is equal to

$$\frac{\partial \ell(w)}{\partial w_i} = \sum_{j=1}^n x_i^j (y^j - p(y^j = 1 | x^j; w)) \quad (14)$$

Hint: does Expression 13 look like a familiar probability?

Solution:

From the previous discussion, we are given that

$$\frac{\partial}{\partial w_i} \sum_{j=1}^n \left[y^j \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right] = \sum_{j=1}^n y^j x_i^j.$$

We notice that Expression 13 can be simplified to $x_i^j p(y^j = 1 | x^j; w)$. Summing over $j = 1$ to n , we have

$$\frac{\partial}{\partial w_i} \sum_{j=1}^n \ln \left[1 + \exp \left(w_0 + w_1 x_1^j + w_2 x_2^j \right) \right] = \sum_{j=1}^n x_i^j p(y^j = 1 | x^j; w)$$

. Therefore, using Equation 9 we conclude that

$$\frac{\partial \ell(w)}{\partial w_i} = \sum_{j=1}^n x_i^j (y^j - p(y^j = 1 | x^j; w)).$$

Since the log likelihood is concave, it is easy to optimize using gradient ascent. The final algorithm is as follows. We pick a step size η , and then perform the following iterations until the change is $< \epsilon$:

$$w_0^{(t+1)} = w_0^{(t)} + \eta \sum_j \left[y^j - p(y^j = 1 | x^j; w^{(t)}) \right] \quad (15)$$

$$w_i^{(t+1)} = w_i^{(t)} + \eta \sum_j x_i^j \left[y^j - p(y^j = 1 | x^j; w^{(t)}) \right]. \quad (16)$$

3.3 The decision boundary [5 points]

Recall the prediction rule for logistic regression is if $p(y^j = 1 | x^j) > p(y^j = 0 | x^j)$, then predict 1, otherwise predict 0. Prove that this is a linear decision boundary using only the prediction rule and the parametric form of the probability.

$$\begin{aligned} & p(y^j = 1 | x^j) > p(y^j = 0 | x^j) \\ \implies & p(y^j = 1 | x^j) > \frac{1}{2} \\ \implies & \frac{1}{1 + \exp(-w_0 - w_1 x_1 - w_2 x_2)} > \frac{1}{2} \\ \implies & \exp(-w_0 - w_1 x_1 - w_2 x_2) < 1 \\ \implies & -w_0 - w_1 x_1 - w_2 x_2 < 0 \\ \implies & w_0 + w_1 x_1 + w_2 x_2 > 0 \end{aligned}$$

Therefore, the decision boundary is a linear separator.